Revision — 2.2.2 Computational Methods

OCR H446 — one-page revision summary

Must know

- A problem is **solvable computationally** if it can be **decomposed**, has clear **inputs/processes/outputs**, is repetitive/rule-based (algorithmic), allows useful **abstractions**, has data that can be **structured/modelled**, and has a clear, **finite** sequence of steps. (Some problems are **non-computable**, e.g. the Halting Problem; some are computable but **intractable**.)
- **Problem recognition** = identifying and understanding what the problem actually is (inputs, outputs, constraints, success criteria) **before** solving it.
- **Decomposition** = breaking a complex problem into smaller, manageable **sub-problems** (usually different sub-tasks).
- **Divide and conquer** = repeatedly splitting a problem into **smaller instances of the same problem**, solving the parts, then **combining** results (binary search, merge/quick sort). It **pairs naturally with recursion** because each part is the same type of problem.
- **Abstraction** = removing/hiding unnecessary detail. **Representational** = drop irrelevant detail (e.g. the Tube map keeps line connections, drops true distances). **By generalisation** = group things by **shared characteristics** so they can be treated the same.
- **Backtracking** = build a solution step by step; on a **dead end**, return to the last decision and try another option (mazes, N-Queens, Sudoku; often recursive).
- **Data mining** = search **large data sets** for hidden patterns, correlations and trends (loyalty-card basket analysis, fraud detection).
- **Heuristics** = a **rule of thumb** giving a **good-enough (not guaranteed optimal)** answer quickly when an exact method is too slow or impossible (satnav A*, Travelling Salesman).
- **Performance modelling** = use a **model** to predict/test how a system behaves before or instead of building it (server load, aircraft systems, algorithm scaling).
- **Pipelining** = split a process into **stages** so different stages work on **different items simultaneously**, raising throughput (CPU fetch–decode–execute).
- **Visualisation** = represent data/problems **graphically** (charts, maps, heat maps) so patterns, trends and outliers are easier to spot and to aid decisions.

Key terms

Term	Definition
Computable problem	One with a clear, finite, rule-based sequence of steps that can be decomposed
Problem recognition	Identifying and understanding the real problem before solving it
Decomposition	Breaking a problem into smaller sub-problems
Divide and conquer	Repeatedly splitting a problem, solving the parts, then combining results
Abstraction	Removing/hiding unnecessary detail to focus on what matters
Representational abstraction	Dropping irrelevant detail while keeping the needed structure
Abstraction by generalisation	Grouping things by shared characteristics
Backtracking	Building a solution step by step, retreating from dead ends
Data mining	Searching large data sets for hidden patterns/correlations
Heuristic	A rule of thumb giving a good-enough (not guaranteed optimal) answer quickly
Performance modelling	Using a model to predict/test system behaviour before building it
Pipelining	Overlapping stages so different items are processed simultaneously
Visualisation	Representing data graphically to reveal patterns and aid decisions
Intractable	Solvable only in exponential time or worse — impractical for large inputs

Worked example / how to — the N-Queens problem (backtracking)

Place 8 queens on a chessboard so no two attack each other:

1. Place queens one at a time, **column by column**, choosing a **safe** square for each.
2. After each placement, check the partial arrangement is still valid (no two queens share a row, column or diagonal).
3. At a **dead end** — no safe square in the next column — **backtrack**: undo the last placement and try the next available square for the previous queen.
4. Repeat until all 8 are placed (a solution) or all options are exhausted.

Why backtracking suits it: it explores possibilities systematically and abandons whole branches as soon as they cannot lead to a solution, avoiding checking every full arrangement. It can still be slow for large boards because the search space grows very quickly (combinatorial explosion).

Exam tips

- Always give a **named example** AND say **why** the method suits that problem — a bare definition rarely earns full marks.
- For **heuristic**, stress *good enough, not guaranteed optimal*, and link it to problems that are **intractable** by exact methods (e.g. TSP).
- For **divide and conquer**, explicitly mention **splitting into same-type sub-problems, solving, then combining**, and that this **pairs with recursion**.
- Keep the two abstractions distinct: **representational** = drop detail; **by generalisation** = group by shared traits.